

Linux Kernel - Study Notes

I. Overview of the Kernel

The Linux operating system consists of **three levels of abstraction**:

1. Hardware Level (most basic)

- CPU, memory, hard disks, networking ports
- Physical layer that performs actual computations

2. Kernel Level (middle layer)

- Handles process and memory management
- Manages device communication
- Handles system calls
- Sets up filesystem
- Acts as intermediary between hardware and user space

3. User Space Level (top layer)

- Shell, programs, graphics
- Applications that users interact with directly

The kernel's primary job is to communicate with hardware to ensure it executes what user processes need.

2. Privilege Levels (Protection Rings)

Why Separate Kernel and User Space?

- **Security**: Prevents malicious software from accessing hardware directly
- **Stability**: Protects system from crashes caused by user applications
- **Control**: Maintains system integrity

Two Main Privilege Levels

- **Ring #0 (Kernel Mode)**
 - Complete hardware access
 - Can execute any system instruction
 - Full trust and control
- **Ring #3 (User Mode)**
 - Limited access to safe memory and CPU

- Restricted operations
- Cannot directly access hardware

Analogy Example

Think of Britney Spears protected by multiple security layers:

- Groupies (outer ring)
- Personal bodyguards (middle ring)
- Bouncer (inner ring)
- Britney herself (innermost ring - highest privilege)

3. System Calls

What Are System Calls?

System calls (syscalls) are the "VIP passes" that allow user space processes to request kernel services.

How System Calls Work

1. User program calls a system call wrapper
2. Wrapper invokes the actual system call
3. System call executes a trap
4. Trap is caught by system call handler
5. Handler references the system call table using syscall ID
6. System switches to kernel mode
7. Kernel executes the requested function
8. Returns to user mode with success/error status

Example

- **stat() system call:** Queries file status
- When `ls` command runs, it uses system calls to access file information

Debugging System Calls

```
strace ls
```

Use `strace` command to view system calls made by a process.

4. Kernel Installation

Check Current Kernel Version

```
$ uname -r  
3.19.0-43-generic
```

Installation Methods

Method 1: Package Manager

```
sudo apt install linux-generic-lts-vivid
```

Then reboot into the new kernel.

Method 2: Specific Version

```
sudo apt install 3.19.0-43-generic
```

Method 3: System-wide Update

```
sudo apt dist-upgrade
```

Updates all packages including kernel.

Note: You can install multiple kernels and choose which one to boot from the GRUB menu.

5. Kernel Location

Files Added to /boot Directory

When installing a new kernel, several files are added to `/boot` :

- **vmlinux**: The actual Linux kernel
- **initrd**: Temporary filesystem used before loading kernel
- **System.map**: Symbolic lookup table
- **config**: Kernel configuration settings for module loading

Maintenance

- Multiple kernel versions create multiple file sets
- Delete old versions if `/boot` runs out of space
- **Warning:** Never delete the currently running kernel files

6. Kernel Modules

Concept

Kernel modules are like car accessories (spoiler, bike rack, hitch) that can be added or removed without changing the core functionality.

Benefits

- Extend kernel functionality without modifying core code
- Load/unload on demand
- Usually no reboot required
- Modular approach to kernel features

Module Management Commands

View Currently Loaded Modules

```
lsmod
```

Load a Module

```
sudo modprobe bluetooth
```

- Loads from `/lib/modules/(kernel version)/kernel/drivers`
- Automatically handles dependencies

Remove a Module

```
sudo modprobe -r bluetooth
```

Module Configuration

Load Module on Bootup

Create configuration file in `/etc/modprobe.d/` :

```
$ /etc/modprobe.d/peanutbutter.conf  
options peanut_butter type=almond
```

Prevent Module from Loading on Bootup

```
$ /etc/modprobe.d/peanutbutter.conf  
blacklist peanut_butter
```

Key Points

- Kernel modules have their own parameters
- Configuration files allow persistent settings
- Blacklisting prevents unwanted modules from loading
- Modules extend functionality without kernel recompilation